stack · v1.1.1

Full Technical Documentation

One skill — and one standalone installer — to install them all.

`stack` installs Longhand, Context-Mode, and Hardgate as a coordinated Claude Code stack — with timestamped config backup, per-tool idempotency guards, and a hardened post-install verification checklist.

Table of Contents

---

1. What This Project Does

Claude Code is a powerful coding assistant, but out of the box it has three significant gaps:

- **No memory** — every session starts from zero. Claude doesn't know what you worked on yesterday.
- **Context flooding** — raw tool output (file reads, bash output, web fetches) pours into the conversation and fills up Claude's context window, causing it to forget earlier parts of the session.
- **No enforcement layer** — Claude can call any tool it decides to use. There's no mechanism to require human approval before risky operations.

`stack` installs three community-built tools that fill exactly these gaps — in the right order, with the right wiring, verified after install.

---

2. Architecture Overview

```
┌──────────────────────────────────────────────────────────────┐
│                        Claude Code session                   │
│                                                              │
│  User prompt                                                 │
│       │                                                      │
│       ▼                                                      │
│  UserPromptSubmit hook                                       │
│  └── Longhand: record prompt to local SQLite + ChromaDB      │
│                                                              │
│  Claude decides to use a tool                                │
│       │                                                      │
│       ▼                                                      │
│  PreToolUse hook (runs twice, in order)                      │
│  ├── Hardgate: check tool against blocked list               │
│  │       blocked? → exit 2 → tool cancelled                  │
│  │       allowed? → continue                                 │
│  └── Context-Mode: run tool in sandbox subprocess            │
│          returns structured summary, not raw output         │
│                                                              │
│  SessionEnd hook                                             │
│  └── Longhand: ingest full session into searchable memory    │
│                                                              │
│  SessionStart hook                                           │
│  └── Hardgate: inject blocked-tool reminder into context     │
└──────────────────────────────────────────────────────────────┘


Config written to: ~/.claude/settings.json
MCP servers:        longhand, context-mode
```

Config File Layout

```
~/.claude/settings.json
├── hooks
│   ├── SessionStart      ← Hardgate reminder
│   ├── UserPromptSubmit  ← Longhand prompt hook
│   ├── PreToolUse        ← Hardgate exit-2 enforcement
│   │                       Context-Mode (Bash, Read, WebFetch + 6 more)
│   └── SessionEnd        ← Longhand session ingest
└── mcpServers
    ├── longhand           ← memory query server
    └── context-mode       ← sandbox execution server
```

---

3. The Three Tools

Longhand

Author: Wynelson94 · github.com/Wynelson94/longhand

Stores every Claude Code session verbatim in SQLite + ChromaDB. Enables semantic recall in ~126ms with zero API cost. Hooks into UserPromptSubmit (to capture prompts as they arrive) and SessionEnd (to ingest the full session after close).

Install command: `pip install longhand`

Artifacts installed:

- SessionEnd hook in `~/.claude/settings.json`
- UserPromptSubmit hook in `~/.claude/settings.json`
- MCP server entry for `longhand mcp-server`

Context-Mode

Author: scottconverse · github.com/scottconverse/context-mode

Sandboxes tool output. Intercepts PreToolUse events for Bash, Read, WebFetch and six additional tool types, runs them in a subprocess sandbox, and returns a structured summary instead of raw output. Keeps raw command results off the context window.

Install command: `git clone` + `node install.js`

Artifacts installed:

- PreToolUse hooks (one per tool type) in `~/.claude/settings.json`
- MCP server entry for context-mode sandbox

Required matchers: Bash, Read, WebFetch (minimum — install adds more)

Hardgate

Author: scottconverse · github.com/scottconverse/hardgate

Blocks forbidden tool calls via `exit 2` hooks. Requires one interactive configuration step (running `/hard-gate` inside a Claude Code session to choose which tools to enforce). After configuration, fires on every PreToolUse event.

Install method: Interactive — requires `/hard-gate` inside an active Claude Code session.

Artifacts installed:

- PreToolUse enforcement hook script
- SessionStart reminder hook in `~/.claude/settings.json`

---

4. Installation — Option A: Standalone Installer (v1.1.0+)

The standalone installer runs without an open Claude Code session. It is the recommended install path.

Requirements

- Python 3.10+
- Node.js 18+
- Claude Code CLI (`claude` on PATH)
- Full `stack` repo cloned locally

Steps

macOS / Linux:

```
git clone https://github.com/scottconverse/stack
cd stack
bash install.sh
```

Windows (any terminal — no Git Bash required):

```
git clone https://github.com/scottconverse/stack
cd stack
python install.py
```

Or double-click `install.bat` .

Missing Dependencies

If Node.js or Claude Code CLI are not found, the installer handles them before proceeding:

- **Node.js missing** — prints the platform-specific install command (`winget install OpenJS.NodeJS` / `brew install node` / `sudo apt install nodejs`) and exits. Re-run after installing.
- **Claude Code CLI missing** — prompts: *Install it now via npm? [Y/n]* — runs `npm install -g @anthropic-ai/claude-code` if confirmed.

What install.py Does

1. Locates `scripts/verify.py` relative to itself
2. Runs pre-flight checks; offers to install missing dependencies
3. Discovers Context-Mode via plugin cache or Python-native home walk (no `find` dependency — works on Windows)
4. Takes timestamped snapshots of `~/.claude/settings.json` and `~/.claude.json`
5. Installs Longhand, then Context-Mode — skips each if already fully present
6. Pauses for interactive Hardgate configuration — skips if already configured
7. Imports `scripts/verify.py` via `importlib.util` and runs the full post-install check suite
8. Exits with the verifier's exit code (0 = all pass, 1 = check failures, 2 = config corruption + auto-restore)

Verify-Only Mode

```
python install.py --verify
```

Skips installation; runs the verifier only. Useful for checking state after manual changes.

5. Installation — Option B: /stack Claude Code Skill

Install from inside an active Claude Code session using the `/stack` skill command.

Requirements

- Same as Option A, plus:
- An active Claude Code session
- bash shell (Git Bash or WSL on Windows)
- `skills/stack.md` added to your Claude Code skill path

Steps

```
git clone https://github.com/scottconverse/stack
# Add skills/stack.md to Claude Code skill path
claude
/stack
```

The skill file instructs Claude Code to perform the same install sequence as `install.py`. Both paths share `scripts/verify.py` for identical post-install verification.

---

6. What Happens During Install

Phase 0 — Verify.py Discovery

Three-stage discovery: env var (`STACK_DIR`) → plugin cache → bounded home directory walk. Both `install.py` and the `/stack` skill use the same discovery logic.

Pre-flight

Checks Python 3.10+, Node 18+, and tool locations. Offers to install missing tools where possible. Takes timestamped snapshots:

```
~/.claude/settings.json.stack-backup-YYYYMMDD-HHMMSS
~/.claude.json.stack-backup-YYYYMMDD-HHMMSS
```

Multiple runs produce independent backups — reruns do not overwrite prior snapshots.

Idempotency

Before installing each tool, the installer checks ALL required artifacts. Skips only if every artifact is fully present. Hardgate is also idempotent — the interactive pause is skipped if Hardgate is already configured.

Install Sequence

1. **Longhand** — `pip install longhand` → `longhand setup` → MCP registration
2. **Context-Mode** — clone → `node install.js` (registers hooks and MCP server)
3. **Hardgate** — interactive pause → `/hard-gate` configuration in Claude Code session (skipped if already done)

Subprocess Output

Subprocess output (pip, node, etc.) is captured and shown only on failure. Successful steps print a one-line summary. Failed steps dump the full captured output with a separator so the error context is preserved.

---

7. Post-Install Verification

`scripts/verify.py` runs automatically after install. Run standalone at any time:

```
python scripts/verify.py
# or
python install.py --verify
```

Check	Level	Retry command
Longhand SessionEnd hook	Required	`longhand setup`
Longhand UserPromptSubmit hook	Required	`longhand prompt-hook install`
Context-Mode PreToolUse hooks	Required	`node install.js` (from context-mode dir)
Longhand MCP server	Required	`claude mcp add longhand -s user -- longhand mcp-server`
Context-Mode MCP server	Required	`node install.js` (from context-mode dir)
Hardgate enforcement	Warning only	`/hard-gate` inside Claude Code

Exit Codes

Code	Meaning	Action
0	All required checks pass	Restart Claude Code
1	One or more checks failed; config intact	Follow retry commands
2	Config file malformed	Both files auto-restored from backup

---

8. Design Decisions

Why a standalone installer alongside the skill? The `/stack` skill requires an open Claude Code session. For new setups or machine migrations where Claude Code isn't running yet, a standalone installer removes that dependency entirely.

Why Python stdlib only in `install.py`? No additional packages to install before running the installer. Reduces friction to zero on a fresh machine.

Why Python-native context-mode discovery? `find` is a Unix command that doesn't exist on Windows without Git Bash. A Python-native `os.walk` with a depth limit works identically on all platforms.

Why share `verify.py` between both install paths? Single source of truth for what "correctly installed" means. A fix to the verifier benefits both paths immediately.

Why capture subprocess output and show only on failure? Pip and Node produce 20–50 lines of output on a normal install. Showing it on every run obscures the signal. On failure, the full output is needed for diagnosis — so it's captured and printed only then.

Why skip Hardgate's interactive pause on re-runs? Re-running the installer (e.g. to fix a partial install) should not force the user back into a Claude Code session to re-configure something that's already working. The idempotency check reads the actual hook wiring, not a flag file.

Why `exit 2` for config corruption (not `exit 1`)? Exit 1 signals "checks failed but config is intact — retry." Exit 2 signals "config is damaged — auto-restore has run." The distinction lets callers handle the two cases differently.

Why return 127 for `FileNotFoundError` in `_run()`? 127 is the POSIX standard exit code for "command not found." Using it here keeps behaviour consistent with shell conventions and distinguishes "command failed" (any non-zero) from "command didn't exist" (127 specifically).

---

9. File Reference

File	Purpose
`install.py`	Standalone installer — Python stdlib only
`install.bat`	Windows double-click launcher for `install.py`
`install.sh`	macOS/Linux launcher for `install.py`
`skills/stack.md`	Claude Code skill file for `/stack` command
`scripts/verify.py`	Post-install state verifier; shared by both install paths
`tests/test_verify.py`	29 unit tests for verifier logic and exit codes
`tests/test_install.py`	30 unit tests for installer (including adversarial cases)
`docs/index.html`	GitHub Pages landing page
`docs/USER-MANUAL.html`	Plain-language user manual
`docs/README-FULL.pdf`	This document

---

10. Tested Versions

Tool	Tested version
Longhand	0.5.5
Context-Mode	1.6.0
Python	3.14.3
Node.js	18+

Pin Longhand if you need a reproducible install:

```
pip install longhand==0.5.5
```

Context-Mode is installed from a local clone — pin by checking out the validated commit hash before running `node install.js`.

---

11. Known Limitations

Config presence ≠ MCP liveness. The verifier checks that config entries are present and correctly structured. It does not verify that MCP servers are actually running and responding. After install, always confirm runtime health:

```
longhand doctor
claude mcp list
```

Hardgate requires a live Claude Code session. The `/hard-gate` configuration step cannot be automated. Both install paths pause and wait for the user to complete it interactively.

Context-Mode discovery depth. The Python-native walk in `install.py` is bounded to 3 directory levels from the home directory. If you've cloned Context-Mode unusually deep in your directory tree, set `CONTEXT_MODE_DIR` to its absolute path before running the installer, or use the plugin cache path (`~/.claude/plugins/cache/context-mode`).

---

stack v1.1.1 · MIT License · github.com/scottconverse/stack